

Universidade Federal de Minas Gerais

Escola de Engenharia

Programa de Pós-Graduação em Engenharia Elétrica

Trabalho Final

Pedro Antonio de Souza Campos Rodrigues

Planejamento de Movimento de Robôs

Prof. Luciano C. A. Pimenta

Belo Horizonte – MG

3 de julho de 2026

Sumário

1	Introdução	2
2	Planejamento de movimento em $SE(3)$	2
2.1	Representação da configuração	2
2.2	Métrica utilizada	2
2.3	Interpolação e avanço	3
2.4	RRT bidirecional	4
2.5	Oráculo de colisão	4
2.6	Discretização de arestas	5
2.7	Pós-processamento por shortcut	5
2.8	Seguimento por campo vetorial	5
2.9	Resumo da implementação	6
3	Resultados preliminares	6
4	Conclusão preliminar	7

1. Introdução

O problema de planejamento de movimento consiste em encontrar uma trajetória livre de colisões entre uma configuração inicial e uma configuração final. Para um corpo rígido livre no espaço, a configuração envolve simultaneamente posição e orientação. Por isso, neste trabalho a configuração do robô é representada em $SE(3)$, o grupo das transformações rígidas no espaço [6].

Métodos baseados em amostragem, como o RRT, são adequados para problemas de planejamento em espaços de configuração de maior dimensão, pois evitam a construção explícita de todo o espaço livre [5, 3]. Neste trabalho, foi implementado um planejador RRT bidirecional em $SE(3)$ para um corpo rígido. O caminho planejado é posteriormente usado como referência para seguimento por campo vetorial.

2. Planejamento de movimento em $SE(3)$

2.1. Representação da configuração

O robô é tratado como um corpo rígido livre. Sua pose é representada por uma matriz homogênea

$$H = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix} \in SE(3),$$

em que $R \in SO(3)$ representa a orientação e $p \in \mathbb{R}^3$ representa a posição. Essa representação permite tratar posição e orientação como uma única configuração do corpo rígido [6, 7, 1].

2.2. Métrica utilizada

Para comparar duas poses H_1 e H_2 , a implementação utiliza o erro relativo

$$H_{\text{rel}} = H_1^{-1} H_2.$$

Esse erro relativo é levado para a álgebra de Lie por meio do logaritmo matricial:

$$\hat{\xi} = \log(H_1^{-1} H_2), \quad \xi = \vee \left(\log(H_1^{-1} H_2) \right).$$

No código, a convenção adotada para o vetor de twist é

$$\xi = \begin{bmatrix} v \\ \omega \end{bmatrix},$$

isto é, a parte linear v vem antes da parte angular ω .

Como o vetor ξ combina componentes translacionais e rotacionais, foi usada a matriz de ponderação

$$G_\ell = \text{diag}(1, 1, 1, \ell^2, \ell^2, \ell^2).$$

O parâmetro ℓ é uma escala de comprimento usada para ponderar a parte angular em relação à parte translacional. A métrica utilizada no planejador é

$$d(H_1, H_2) = \sqrt{\xi^\top G_\ell \xi}.$$

Essa métrica foi usada para a busca do nó mais próximo, para o avanço do planejador, para a checagem e discretização de arestas e para a discretização do caminho final. A necessidade de ponderar translação e rotação em movimentos rígidos é discutida por Park [8]; a formulação por twists, exponencial e logaritmo segue a representação usual de $SE(3)$ [6, 1].

2.3. Interpolação e avanço

A interpolação entre duas poses é calculada diretamente em $SE(3)$ por

$$H(s) = H_1 \exp \left(s \log(H_1^{-1} H_2) \right), \quad s \in [0, 1].$$

Essa interpolação utiliza o movimento relativo entre as poses e evita a parametrização da orientação por ângulos de Euler [6, 1].

O operador de avanço usado pelo RRT segue a mesma construção. Dado um nó próximo H_{near} e uma pose alvo H_{target} , a nova pose é definida por

$$H_{\text{new}} = H_{\text{near}} \exp \left(\alpha \log(H_{\text{near}}^{-1} H_{\text{target}}) \right),$$

com

$$\alpha = \min \left(1, \frac{\eta}{d(H_{\text{near}}, H_{\text{target}})} \right).$$

Se o alvo está dentro do passo máximo, o novo nó coincide com o alvo. Caso contrário, o novo nó avança apenas até a distância máxima η , medida pela métrica definida anteriormente.

2.4. RRT bidirecional

O planejador implementado é um RRT bidirecional com estratégia do tipo RRT-Connect. São mantidas duas árvores: T_{start} , inicializada em H_{start} , e T_{goal} , inicializada em H_{goal} . A cada iteração, uma árvore é expandida e a outra é usada como referência para tentativa de conexão.

Algorithm 1 RRT bidirecional em $SE(3)$

Require: H_{start} , H_{goal} , limites de amostragem, métrica $d(\cdot, \cdot)$, passo η

Ensure: Caminho discretizado livre de colisões ou falha

```

1: Inicializar  $T_{\text{start}}$  com  $H_{\text{start}}$ 
2: Inicializar  $T_{\text{goal}}$  com  $H_{\text{goal}}$ 
3: for  $k = 1, \dots, N_{\text{max}}$  do
4:   Selecionar a árvore ativa  $T_a$  e a árvore oposta  $T_b$ 
5:   Amostrar uma configuração  $H_{\text{rand}}$  em  $SE(3)$ 
6:   Encontrar  $H_{\text{near}} \in T_a$  mais próximo de  $H_{\text{rand}}$ 
7:   Calcular  $H_{\text{new}}$  pelo avanço em  $SE(3)$ 
8:   if  $H_{\text{new}}$  é livre e a aresta  $(H_{\text{near}}, H_{\text{new}})$  é livre then
9:     Adicionar  $H_{\text{new}}$  em  $T_a$ 
10:    Tentar conectar  $T_b$  a  $H_{\text{new}}$ 
11:    if as árvores foram conectadas then
12:      Reconstruir o caminho entre  $H_{\text{start}}$  e  $H_{\text{goal}}$ 
13:      Aplicar pós-processamento por shortcut
14:      Discretizar o caminho final
15:      return caminho discretizado
16:    end if
17:  end if
18:  Alternar os papéis de  $T_a$  e  $T_b$ 
19: end for
20: return falha

```

Na implementação, a amostragem possui termos de viés para favorecer a conexão entre as árvores. Com uma certa probabilidade, o alvo da expansão é escolhido como um nó da árvore oposta; com outra probabilidade, é escolhida a raiz da árvore oposta. Caso contrário, uma nova configuração é amostrada no espaço de busca. A posição é amostrada em uma caixa limitada e a orientação é amostrada em $SO(3)$. Essa estrutura segue os princípios de RRT e RRT-Connect para planejamento de movimento [5, 3, 4].

2.5. Oráculo de colisão

O robô é representado por primitivas geométricas no frame local do corpo. Para uma pose H , cada primitiva local é transformada para o mundo por

$$H_{\text{obj}} = HH_{\text{local}}.$$

Depois da transformação, calcula-se a AABB de cada primitiva. Essas caixas são usadas como etapa de broad phase, de modo que apenas pares candidatos sejam avaliados pela rotina de distância geométrica. A pose é considerada em colisão quando a distância entre primitivas é menor que uma tolerância d_{tol} .

2.6. Discretização de arestas

Verificar apenas os nós da árvore não basta, pois o movimento entre duas poses livres pode cruzar obstáculos. Assim, cada aresta é discretizada de acordo com a métrica em $SE(3)$:

$$n = \left\lceil \frac{d(H_1, H_2)}{\Delta_{edge}} \right\rceil.$$

As poses intermediárias são geradas por

$$H_i = H_1 \exp \left(s_i \log(H_1^{-1} H_2) \right), \quad s_i = \frac{i}{n}.$$

O parâmetro Δ_{edge} é usado para a checagem de colisão ao longo das arestas. Já Δ_{out} é usado apenas para discretizar o caminho final retornado.

2.7. Pós-processamento por shortcut

O caminho bruto produzido pelo RRT pode conter desvios desnecessários. Por isso, foi implementado um pós-processamento por shortcut. O procedimento escolhe dois waypoints do caminho e testa a conexão direta entre eles. Se essa conexão direta estiver livre de colisão, os waypoints intermediários são removidos. Esse procedimento tende a reduzir o caminho, mas não garante otimalidade global [5, 3].

2.8. Seguimento por campo vetorial

Depois do planejamento, o caminho discretizado é usado como curva de referência para um campo vetorial em $SE(3)$. O campo vetorial fornece diretamente um twist desejado

$$\xi_d = \begin{bmatrix} v_d \\ \omega_d \end{bmatrix},$$

que é aplicado ao modelo cinemático do corpo rígido. Essa abordagem é compatível com formulações de seguimento de caminhos em grupos de Lie, como discutido por Bartelt et al. [2], especialmente em $SE(3)$, onde a entrada pode ser interpretada como o twist mecânico do corpo.

De forma geral, o campo vetorial combina uma componente de convergência para a curva com uma componente de avanço ao longo da curva. No código, o twist retornado pelo campo vetorial é modulado por um fator escalar α , definido em função da distância até a pose final:

$$\alpha(d) = 1 - e^{-\lambda d} (1 + \lambda d),$$

em que

$$d = \left\| \log \left(H^{-1} H_{\text{goal}} \right) \right\|.$$

Quando o corpo está longe do alvo, $\alpha(d)$ fica próximo de 1, de modo que o campo vetorial praticamente não é alterado. Quando o corpo se aproxima da pose final, $\alpha(d)$ tende a 0, reduzindo gradualmente o twist desejado. Isso suaviza a chegada e evita que o robô continue avançando com velocidade significativa perto da pose final. O twist aplicado é

$$\xi = \alpha(d) \xi_d.$$

2.9. Resumo da implementação

O núcleo do planejador foi implementado em C++ e é chamado por uma interface Python. O resultado retorna sucesso ou falha, caminho por waypoints, caminho discretizado, número de iterações, número de nós nas árvores e tempos de execução. O caminho discretizado é usado pelo campo vetorial para gerar o movimento do corpo rígido.

3. Resultados preliminares

O cenário considerado contém um corpo rígido representando um UAV. Os obstáculos são compostos por caixas e cilindros. O RRT bidirecional foi usado para conectar uma pose inicial a uma pose alvo em $SE(3)$. O caminho retornado pelo planejador foi pós-processado por shortcut e, em seguida, discretizado para ser usado como referência pelo campo vetorial. A simulação em UAIBot mostra o caminho planejado e o caminho seguido pelo corpo rígido.

Para avaliar o tempo de execução do planejador, foi realizado um benchmark com 100 execuções independentes do mesmo problema de planejamento. Não foi fixada semente aleatória, de modo que as variações observadas refletem a natureza amostral do RRT. Em todas as execuções, foram mantidos os mesmos parâmetros de planejamento, o mesmo modelo geométrico do corpo rígido e o mesmo conjunto de obstáculos.

Das 100 execuções realizadas, o planejador obteve sucesso em 100 casos e não apresentou

falhas. Assim, a taxa de sucesso observada no cenário testado foi de

$$\frac{100}{100} = 100,00\%.$$

Considerando apenas as execuções bem-sucedidas, o tempo médio de execução foi

$$\bar{t}_{\text{exec}} = 0,022736 \text{ s},$$

com desvio padrão

$$\sigma_t = 0,010538 \text{ s}.$$

Em milissegundos, esses valores correspondem aproximadamente a 22,736 ms de média e 10,538 ms de desvio padrão.

Os resultados indicam que, para o cenário considerado, o planejador foi capaz de encontrar caminhos livres de colisão de forma consistente. A variação no tempo de execução é esperada, pois o RRT bidirecional é baseado em amostragem aleatória. Assim, diferentes execuções podem gerar árvores distintas e encontrar conexões em momentos diferentes da busca.

Essa avaliação é preliminar e tem como objetivo caracterizar o comportamento do planejador no cenário implementado. Uma análise mais ampla poderia considerar outros ambientes, diferentes resoluções de checagem de arestas e diferentes parâmetros de amostragem.

4. Conclusão preliminar

Foi implementado um pipeline de planejamento para corpo rígido em $SE(3)$. A métrica logarítmica ponderada permitiu combinar posição e orientação na busca, e o RRT bidirecional encontrou um caminho livre de colisões no cenário avaliado. A discretização de arestas foi usada para verificar o movimento entre poses, e o pós-processamento por shortcut reduziu desvios do caminho bruto.

O caminho discretizado foi usado como referência para seguimento por campo vetorial. A modulação do twist desejado reduziu a velocidade próxima ao alvo, suavizando a chegada à pose final. O método implementado, portanto, integra planejamento em $SE(3)$, verificação de colisão e seguimento de caminho para um corpo rígido livre.

Referências

- [1] Timothy D. Barfoot. *State Estimation for Robotics*. Cambridge University Press, 2017.

- [2] Felipe Bartelt, Vinicius M. Gonçalves, and Luciano C. A. Pimenta. Constructive vector fields for path following in fully-actuated systems on matrix lie groups, 2026.
- [3] Howie Choset, Kevin M. Lynch, Seth Hutchinson, George A. Kantor, Wolfram Burgard, Lydia E. Kavraki, and Sebastian Thrun. *Principles of Robot Motion: Theory, Algorithms, and Implementations*. MIT Press, 2005.
- [4] James J. Kuffner and Steven M. LaValle. Rrt-connect: An efficient approach to single-query path planning. In *Proceedings of the IEEE International Conference on Robotics and Automation*, pages 995–1001, 2000.
- [5] Steven M. LaValle. *Planning Algorithms*. Cambridge University Press, 2006.
- [6] Kevin M. Lynch and Frank C. Park. *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press, 2017.
- [7] Richard M. Murray, Zexiang Li, and S. Shankar Sastry. *A Mathematical Introduction to Robotic Manipulation*. CRC Press, 1994.
- [8] Frank C. Park. Distance metrics on the rigid-body motions with applications to mechanism design. *Journal of Mechanical Design*, 117(1):48–54, 1995.